

AI-Assisted Theorem Proving

Computer-Assisted Theorem Proving (7CCSACTP)

Mohammad Abdulaziz

Where We Are in the Course

■ Covered in Weeks 1–4:

- **Week 1:** ITP fundamentals and the Isabelle/HOL environment.
- **Week 2:** Mathematical induction and induction hypothesis generalisation.
- **Week 3:** List recursion, structural induction, and proof automation (`simp`, `auto`, `nitpick`).
- **Week 4:** Algebraic datatypes, termination size measures, and tree induction.

This Week

- Today I will introduce you to the last and most recent part of computer-assisted theorem provers' foundations: **AI-assisted theorem proving**.

This Week

- Today I will introduce you to the last and most recent part of computer-assisted theorem provers' foundations: **AI-assisted theorem proving**.
- **Two types covered:**
 1. **Classical machine-learning based approaches:**
 - ▶ Premise selection & external automated provers (Sledgehammer).
 2. **Modern machine learning-based approaches:**
 - ▶ Reinforcement learning (RL) loops with compiler verifiers (AlphaProof, Aristotle).
 - ▶ Off-the-shelf coding agents via Model Context Protocol (MCP).
 - ▶ Two-stage Draft, Sketch, and Prove (DSP) architectures.

1. Sledgehammer

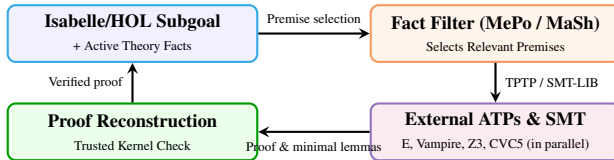
■ First & Most Impactful Automation Attempt:

- "Sledgehammer" in Isabelle/HOL [Paulson and Blanchette, 2010] (Paulson, Blanchette et al.).
- Bridges interactive theorem proving with external ATPs and SMT solvers.

1. Sledgehammer

■ First & Most Impactful Automation Attempt:

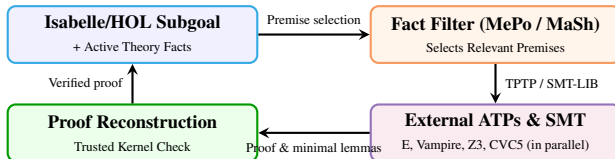
- "Sledgehammer" in Isabelle/HOL [Paulson and Blanchette, 2010] (Paulson, Blanchette et al.).
- Bridges interactive theorem proving with external ATPs and SMT solvers.



1. Sledgehammer

■ First & Most Impactful Automation Attempt:

- **"Sledgehammer"** in Isabelle/HOL [Paulson and Blanchette, 2010] (Paulson, Blanchette et al.).
- Bridges interactive theorem proving with external ATPs and SMT solvers.



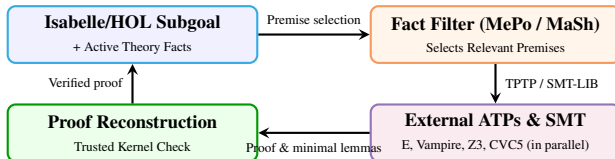
■ Core Workflow & Reconstruction:

- **Fact Filter:** Selects relevant lemmas via heuristic/ML filters (MePo, MaSh).
- **Kernel Soundness:** External proofs are reconstructed in Isabelle (preventing soundness bugs from external provers).

1. Sledgehammer

■ First & Most Impactful Automation Attempt:

- "Sledgehammer" in Isabelle/HOL [Paulson and Blanchette, 2010] (Paulson, Blanchette et al.).
- Bridges interactive theorem proving with external ATPs and SMT solvers.



■ Core Workflow & Reconstruction:

- **Fact Filter:** Selects relevant lemmas via heuristic/ML filters (MePo, MaSh).
- **Kernel Soundness:** External proofs are reconstructed in Isabelle (preventing soundness bugs from external provers).

■ Demo!

2. Custom Models for Formal Proving

■ Key Players & Breakthroughs:

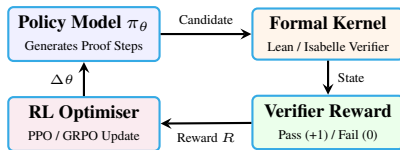
- **DeepMind** (AlphaProof), **Harmonic** (Aristotle), **Axiom Math** (AxiomProver).
- **IMO Gold** achieved in 2025; **PutnamBench 2025** solved by ≥ 20 proving systems.

2. Custom Models for Formal Proving

■ Key Players & Breakthroughs:

- **DeepMind** (AlphaProof), **Harmonic** (Aristotle), **Axiom Math** (AxiomProver).
- **IMO Gold** achieved in 2025; **PutnamBench 2025** solved by ≥ 20 proving systems.

■ Custom RL Loop with Formal Verifiers:

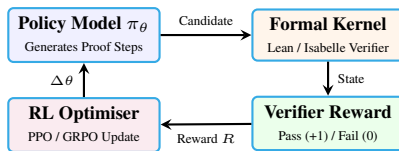


2. Custom Models for Formal Proving

■ Key Players & Breakthroughs:

- **DeepMind** (AlphaProof), **Harmonic** (Aristotle), **Axiom Math** (AxiomProver).
- **IMO Gold** achieved in 2025; **PutnamBench 2025** solved by ≥ 20 proving systems.

■ Custom RL Loop with Formal Verifiers:



■ Problem: Inaccessibility & Prohibitive Costs

- Requires dedicated GPU clusters and multi-million-dollar training budgets.
- **Pre-training / RL Stage**: \$5M – \$100M+ base / \$250k – \$1M+ RL fine-tuning.
- **Inference / Test-Time Search**: \$60 – \$300+ per solved theorem.

3. Democratisation via Agentic Proving

■ Off-the-Shelf Coding Agents (Batch Loop):

- Josef Urban et al. showed standard coding agents (e.g. *Claude Code*) formalised an entire textbook (*Munkres Topology*, 800+ theorems) using simple `isabelle build` for $\approx$ \$200 / month.

3. Democratisation via Agentic Proving

■ Off-the-Shelf Coding Agents (Batch Loop):

- Josef Urban et al. showed standard coding agents (e.g. *Claude Code*) formalised an entire textbook (*Munkres Topology*, 800+ theorems) using simple `isabelle build` for $\approx$ \$200 / month.

■ Follow-up: Interactive Proving via MCP:

- Subsequent work connects agents directly via **Model Context Protocol (MCP)**.
- Replaces batch CLI loops with rich, interactive kernel feedback (no cluster training needed).

3. Democratisation via Agentic Proving

■ Off-the-Shelf Coding Agents (Batch Loop):

- Josef Urban et al. showed standard coding agents (e.g. *Claude Code*) formalised an entire textbook (*Munkres Topology*, 800+ theorems) using simple `isabelle build` for $\approx$ \$200 / month.

■ Follow-up: Interactive Proving via MCP:

- Subsequent work connects agents directly via **Model Context Protocol (MCP)**.
- Replaces batch CLI loops with rich, interactive kernel feedback (no cluster training needed).

■ Practical Advantages:

- No specialised GPU infrastructure or training required.
- Fast, interactive feedback loop directly with the Isabelle/HOL environment.

4. Technical Setup: Agentic LLMs for Theorem Proving

■ Integrating Theorem Provers with LLMs:

- Connect a coding agent (e.g. *Claude Code*, *Open Code*, *Codex*, *Agy*) directly to Isabelle.

4. Technical Setup: Agentic LLMs for Theorem Proving

■ Integrating Theorem Provers with LLMs:

- Connect a coding agent (e.g. *Claude Code*, *Open Code*, *Codex*, *Agy*) directly to Isabelle.

■ Model Context Protocol (MCP) Architecture:



4. Technical Setup: Agentic LLMs for Theorem Proving

■ Integrating Theorem Provers with LLMs:

- Connect a coding agent (e.g. *Claude Code*, *Open Code*, *Codex*, *Agy*) directly to Isabelle.

■ Model Context Protocol (MCP) Architecture:



■ MCP Implementations & Repositories:

- Programmatic actions: `write-file`, `process-file`, `save-file`, etc.
- **PIDE-MCP**: github.com/kappelman/isabelle-pide-mcp
- **Isabelle/Q**: github.com/aws-labs/AutoCorrode (AWS Labs)

4. Technical Setup: Agentic LLMs for Theorem Proving

■ Integrating Theorem Provers with LLMs:

- Connect a coding agent (e.g. *Claude Code*, *Open Code*, *Codex*, *Agy*) directly to Isabelle.

■ Model Context Protocol (MCP) Architecture:



■ MCP Implementations & Repositories:

- Programmatic actions: `write-file`, `process-file`, `save-file`, etc.
- **PIDE-MCP**: github.com/kappelman/isabelle-pide-mcp
- **Isabelle/Q**: github.com/aws-labs/AutoCorrode (AWS Labs)

■ Demo!

5. Agentic Proving Architectures

■ Basic Two-Stage Architecture [Jiang et al., 2023]:



5. Agentic Proving Architectures

■ Basic Two-Stage Architecture [Jiang et al., 2023]:



■ Two-Stage Workflow & Feedback Loop:

- **Planner / Sketcher:** Produces high-level proof sketches (`sorry`) [Jiang et al., 2023].
- **Coder:** Fills in intermediate Isar proof steps & verifies against Isabelle via MCP.
- **Error Loop:** Unresolved errors pass back to the Planner with diagnostics.

5. Agentic Proving Architectures

■ Basic Two-Stage Architecture [Jiang et al., 2023]:



■ Two-Stage Workflow & Feedback Loop:

- **Planner / Sketcher:** Produces high-level proof sketches (`sorry`) [Jiang et al., 2023].
- **Coder:** Fills in intermediate Isar proof steps & verifies against Isabelle via MCP.
- **Error Loop:** Unresolved errors pass back to the Planner with diagnostics.

■ Further Reading & LGT Discussion:

- A paper on **KEATS** will be provided for reading and discussion in the **LGT**.

6. Current State & Industrial Adoption

■ Specialised Startups:

- Companies developing formal reasoning tools: *Harmonic*, *Math Inc*, *Axiom Math*.

6. Current State & Industrial Adoption

■ Specialised Startups:

- Companies developing formal reasoning tools: *Harmonic*, *Math Inc*, *Axiom Math*.

■ Formal Verification in Industry:

- **Apple:** Post-quantum cryptography verification in *corecrypto* via AutoCorres2.
- **AWS:** Automated verification for hypervisors (*Nitro Hypervisor*) and cloud security policies.
- **Arm & Intel:** Hardware, firmware, and instruction set architecture verification.

6. Current State & Industrial Adoption

■ Specialised Startups:

- Companies developing formal reasoning tools: *Harmonic*, *Math Inc*, *Axiom Math*.

■ Formal Verification in Industry:

- **Apple:** Post-quantum cryptography verification in *corecrypto* via AutoCorres2.
- **AWS:** Automated verification for hypervisors (*Nitro Hypervisor*) and cloud security policies.
- **Arm & Intel:** Hardware, firmware, and instruction set architecture verification.

■ Mathematical Research & Breakthroughs:

- **Terence Tao:** Integrating autoformalization and agentic provers into active research.
- **OpenAI (Astra):** Solved 10 open problems with machine-checked Lean certificates.
- **Claude:** Discovered Hadamard matrices for all open orders < 2000 (e.g. order 668).
- **Axiom Math:** Proved prime gap bound ≤ 212 (reduced from 246) in Lean 4.

7. Closing Remarks

■ Core Skills You Must Master Yourself:

1. Splitting problems into manageable formal subproblems.
2. Formally modelling problem domains.
3. Formulating mathematically sound, correct specifications.
4. Constructing correct, readable, concise, and fast-checking proofs (Isabelle linter).

References I

- Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs, February 2023.
- Lawrence C Paulson and Jasmin Christian Blanchette. Three years of experience with sledgehammer, a practical link between automatic and interactive theorem provers. In *PAAR@ IJCAR*, pages 1–10, 2010.